

PISSIR – Connected Games Platform

Final Project – Technical Report

Game of choice: Breakout (Arkanoid)

Pau Reverte Garcia
Academic year 2025/2026
Alberto Testa

INDEX

1. Functional Specifications.....	2
1.1 Description of the application domain	2
1.2 Required functionalities	2
Local Control (Edge)	2
Accessibility via Internet (Server)	2
1.3 Main use cases.....	2
2. Technological Analysis.....	3
2.1 Network and context restrictions.....	3
2.2 Alternatives analyzed.....	3
3. Choice of Technological Approach.....	4
3.1 Reason for choosing REST + MySQL.....	4
3.2 Persistence architecture adopted	4
4. Software Architecture	5
4.1 Modules and functionalities.....	5
4.2 Concurrent Activities (Threads)	5
5. Implementation Description.....	6
5.1 Implemented classes	6
BreakoutGame (game/BreakoutGame.java)	6
BallThread (game/BallThread.java).....	6
GameController (api/GameController.java)	6
EdgeStorageService (edge/EdgeStorageService.java)	6
SyncService (edge/SyncService.java).....	6
UserService (auth/UserService.java)	6
5.2 User interface	6
6. Validation	7
6.1 Definition of tests	7
6.2 Results	7
7. Conclusions	8

1. Functional Specifications

1.1 Description of the application domain

The game chosen is Breakout, inspired by the classic Arkanoid. The player controls a horizontal paddle to bounce a ball and destroy the blocks located at the top of the screen. The way to win is to destroy all the blocks in the level without the ball falling more than 3 times. The system implements three difficulty levels (Easy, Normal and Hard) and supports multiple simultaneous instances on the same computer, to be able to simulate what a current program could be on an arcade machine, for example.

The main feature of the project is the Edge (Offline-First) capability: the game continues to function fully even if the cloud MySQL server is unavailable. When the connection is restored, the SyncService automatically uploads the locally saved games.

1.2 Required functionalities

Local Control (Edge)

- Simulation of ball physics in a dedicated thread (BallThread) at ~20 ticks/second.
- Collision management: ball with blocks, racket and walls.
- Offline mode: games are saved to a local JSON file (edge_data_<uuid>.json) if the server is not accessible.
- Multi-instance support: each instance uses a different port (port 0 if 8080 is busy).
- Three configurable difficulty levels: Easy, Normal and Hard.
- Random power-ups, between x2 score or an extra ball, that drop when destroying certain blocks.

Accessibility via Internet (Server)

- Full REST API to control the game remotely (VisualStudioCode, Postman, etc.).
- Connection to MySQL to persist scores and users.
- Automatic synchronization: SyncService checks every 2 seconds if there is local data pending to be uploaded.
- Web ranking: HTML/JS page to check the best scores.
- User login system: registration and basic authentication.

1.3 Main use cases

- CU1 – Play locally: the player starts a game, the ball moves in the BallThread and collisions are updated in real time.
- CU2 – End game: the score is saved to MySQL or local JSON if there is no connection.
- CU3 – Synchronize offline: when MySQL returns, the SyncService detects the local JSON and uploads the pending items.
- CU4 – Check ranking: the customer accesses the website to see the best global scores.
- CU5 – API control: the player sends HTTP requests to move the racket or check the game status.
- CU6 – Multi-instance: Multiple instances of the game open on the same PC without port conflicts.

2. Technological Analysis

2.1 Network and context restrictions

The system must operate on private networks without port forwarding, but must be able to connect to a remote MySQL server to synchronize scores. The traffic generated is of the request-response (REST) type, with small message sizes and acceptable latency of up to 1 second for score synchronization.

The main restriction is the intermittent availability of the server: the game cannot depend on constant connection, hence the need for the Edge (Offline-First) pattern with later synchronization.

2.2 Alternatives analyzed

Technology	Advantages	Disadvantages
REST/HTTP (chosen)	Easy to implement, standard, well supported by Spring Boot	Not real-time, requires polling for ranking
WebSocket	Bi-directional, ideal for real-time notifications	Added complexity for a single turn game
MQTT	Very lightweight, pub/sub, offline-resistant	Requires dedicated broker, less standard for web games
gRPC	Low overhead, strong typing	High learning curve, less browser compatible

3. Choice of Technological Approach

3.1 Reason for choosing REST + MySQL

I chose a REST API with Spring Boot and MySQL as the central backend for the following reasons:

- **Simplicity:** REST is the best known and documented paradigm for web applications, and Spring Boot allows you to generate a functional server with very little configuration code.
- **Compatibility:** The graphical interface and the ranking page, HTML/JS, can consume the same API without additional dependencies.
- **Horizontal scalability:** If necessary, multiple server instances can be deployed behind a load balancer without changing the client.
- **Integrated offline mode:** thanks to the Edge pattern, the client can work without the server and synchronize when the connection returns.

3.2 Persistence architecture adopted

Two complementary persistence layers are used:

- **MySQL (online):** main storage for users, games and global ranking. Accessed via Spring Data JPA.
- **Local JSON (offline):** `edge_data_<uuid>.json` file for each instance. Each instance generates a random UUID to avoid collisions between multiple instances on the same PC.
- **SyncService (@Scheduled, every 2 seconds):** Checks for data in the local JSON and uploads it to MySQL when the connection returns.

4. Software Architecture

4.1 Modules and functionalities

Package / Component	Main classes	Responsibility
game	BreakoutGame, Ball, Paddle, Block, BallThread	Game logic, ball physics, collisions, power-ups
celery	GameController, UserController, RankingController	REST API: endpoints to create/control games, login and ranking
persistence	GameRepository, UserRepository (JPA)	Access to MySQL via Spring Data JPA
edge	EdgeStorageService, SyncService	Local JSON persistence and automatic synchronization
auth	UserService, UserModel	Registration, login, session management (loggedIn)
oh	BreakoutPanel (Swing)	Classic graphical interface with Swing

4.2 Concurrent Activities (Threads)

The system uses the following concurrency model:

- BallThread (Runnable): executes the game loop. Separates the physics simulation from the REST API to avoid crashes.
- SyncService (@Scheduled): secondary thread that pings MySQL every 2 seconds and uploads pending items to the local JSON.
- Spring Boot HTTP threads: concurrent management of REST requests for multiple simultaneous instances.
- Port management: if port 8080 is busy, Spring Boot assigns port 0 (random free port of the OS) to support multi-instance.

5. Implementation Description

5.1 Implemented classes

BreakoutGame (game/BreakoutGame.java)

Contains the overall state of a game: list of blocks, position of the racket and ball, score and difficulty level. Main methods:

- `initGame(difficulty)`: initializes ball, racket and block coordinates according to the difficulty.
- `getState()`: Returns the current state serializable in JSON for the REST API.
- `applyPaddleMove(direction)`: updates the paddle position in a thread-safe (synchronized) way.

BallThread (game/BallThread.java)

Implement `Runnable`. Execute the game loop: move the ball, check for collisions, and update state. `Thread.sleep` ensures that the CPU doesn't get saturated and that the REST API can respond while the game is running. When it detects game over (ball lost or all blocks destroyed), it notifies the `GameController` via callback.

GameController (api/GameController.java)

Spring Boot REST Controller. Main endpoints:

- `POST /api/games/start`: Creates a new game, instantiates `BallThread` and returns a `gameId` (hexadecimal hash).
- `GET /api/games/{gameId}/state`: Returns the current state of the game in JSON.
- `POST /api/games/{gameId}/move`: move racket (left/right).
- `POST /api/games/{gameId}/end`: ends the game and saves the score.

EdgeStorageService (edge/EdgeStorageService.java)

Manages the local JSON file `edge_data_{<uuid>.json`. Each instance of the game generates a unique UUID on startup to avoid collisions between multiple instances on the same PC. Provides methods for saving and reading games that are pending synchronization.

SyncService (edge/SyncService.java)

Spring service with `@Scheduled (fixedDelay = 2000 ms)`. On each execution: first checks if MySQL is accessible by doing a light ping, then if it is accessible and there is data in the local JSON, uploads it to MySQL via `GameRepository` and empties the local file.

UserService (auth/UserService.java)

Manages user registration and login. The user model includes username, profile image in Base64 and a boolean `loggedIn` to avoid simultaneous sessions with the same account. Passwords are stored in plain text (own and academic decision to achieve simplicity; in production BCrypt would be required).

5.2 User interface

The project offers two complementary interfaces:

- Swing GUI (`BreakoutPanel`): Classic graphical interface with real-time visualization of the ball, racket and blocks. Allows keyboard control.
- Web ranking (HTML/JS): static page that queries the `GET /api/ranking` endpoint via `fetch()` and displays the top 10 scores.

6. Validation

6.1 Definition of tests

The tests are implemented with JUnit 5 and Spring Boot Test and cover the main scenarios of the system:

ID Test	Scenario	Expected result
T1	Initial state of the game	Ball and racket in starting position, score == 0
T2	Ball-left wall collision	Speed x reversed, ball does not go out of bounds
T3	Ball-racket collision	Speed and reverse, continuous play
T4	The ball falls below the racket	GameState == FINISHED, score recorded
T5	Destruction of all blocks	GameState == FINISHED (win)
T6	Difficulty Easy vs Hard	Correct racket speeds and sizes for each level
T7	Offline persistence (without MySQL)	Game saved to local JSON without exception
T8	Automatic synchronization	Local JSON flushed and data in MySQL on reconnect
T9	Multi-instance (+2 simultaneous games)	Each instance on a different port, without conflicts
T10	Single login per user (loggedIn)	Second login rejected if the user already has an active session
T11	API REST POST /api/games/start	Returns valid gameId and HTTP code 200
T12	API REST POST /api/games/{id}/move	Racket updated successfully on the next tick

6.2 Results

All 12 tests pass successfully. In particular:

- T7 and T8 verify Edge behavior: without MySQL, the matches are saved to local JSON without any user-visible exceptions; when MySQL returns, the SyncService uploads them in less than 2 seconds.
- T9 verifies multi-instance support: two instances of the game run simultaneously on the same PC without port or JSON file conflicts (thanks to unique UUIDs per instance).
- T10 verifies that the basic session system (loggedIn) prevents two simultaneous logins with the same account.

7. Conclusions

The development of the platform has successfully achieved all the functional and technological objectives initially set out in the practice guide. The architecture based on the Edge approach has proven to be a highly efficient solution to ensure system resilience, as the combination of a temporary local JSON file with the asynchronous synchronization service protects the user experience against crashes of the central data server without suffering any loss of information.

Likewise, the design has robustly managed the challenges of concurrency thanks to the use of a thread dedicated exclusively to ball collisions, which decouples the simulation of network performance and the REST API. This openness of the game through HTTP access points also allows the use of external control mechanisms such as cURL or Postman.

Finally, dynamic port reconfiguration has solved data trampling in multi-instance environments, demonstrating that, despite the academic commitments assumed in terms of security due to time constraints, a fully functional and interconnected distributed prototype has been achieved.

As a continuation of this project, the following technological improvements are proposed:

1. Security: Implement Spring Security and password encryption using the BCrypt algorithm with JWT tokens to secure the REST API.
2. Local data migration: Replace individual JSON files with an embedded and thread-safe local database such as SQLite in file mode, improving offline transactional control.
3. Interface modernization: Evolve the Java Swing interface towards a rich web client architecture connected by WebSockets in order to receive game data reactively in real time.